

Грамматика

Терминалы

Пробельные символы, кроме перевода строки после определения, считаются разделителями и не входят в грамматику. Ключевое слово `let` не считается переменной.

LET	::= "let"
LAMBDA	::= "\"
EQUALS	::= "="
DOT	::= "."
LPAREN	::= "("
RPAREN	::= ")"
NEWLINE	::= "\n"
EOF	::= end of input
VARIABLE	::= LETTER { LETTER DIGIT "_" }
LETTER	::= "a".. "z" "A".. "Z"
DIGIT	::= "0".. "9"

Факторизованная EBNF-грамматика

Грамматика описывает набор именованных определений и одно основное лямбда-выражение. Лямбда-абстракции с несколькими параметрами разрешены: `\x y z. expr` означает `\x.\y.\z.expr`.

program	::= { definition } expression EOF
definition	::= LET VARIABLE EQUALS expression NEWLINE
expression	::= abstraction application
abstraction	::= LAMBDA variable-list DOT expression
variable-list	::= VARIABLE { VARIABLE }
application	::= operand { operand } [abstraction]
operand	::= VARIABLE LPAREN expression RPAREN

В записи используются EBNF-операторы $\{\dots\}$ и $[\dots]$; отдельных правил вида $A ::= \epsilon$ в грамматике нет. Для парсера это означает обычные циклы и проверку следующего токена.

Примеры вывода

Пример 1

Вход:

```
let S = \x y z.x z (y z)
let K = \x y.x
S K K
```

Вывод по грамматике:

```
program
=> { definition } expression EOF
=> definition definition expression EOF
=> let S = expression NEWLINE let K = expression NEWLINE expression EOF
=> let S = abstraction NEWLINE let K = abstraction NEWLINE application
    EOF
=> let S = \ variable-list . expression NEWLINE let K = \ variable-list
    . expression NEWLINE S K K EOF
=> let S = \ x y z . application NEWLINE let K = \ x y . application
    NEWLINE S K K EOF
=> let S = \ x y z . x z (y z) NEWLINE let K = \ x y . x NEWLINE S K K
    EOF
```

После подстановки определений и бета-редукции нормальная форма:

```
\x.x
```

Пример 2

Вход:

```
x \y.y
```

Вывод по грамматике:

```
program
=> { definition } expression EOF
=> expression EOF
=> application EOF
=> operand [ abstraction ] EOF
=> x abstraction EOF
=> x \ variable-list . expression EOF
=> x \ y . application EOF
=> x \ y . operand EOF
=> x \ y . y EOF
```

Это выражение читается как:

x (\y.y)

Пример 3

Вход:

(\x.x) y

Вывод по грамматике:

```
program
=> { definition } expression EOF
=> expression EOF
=> application EOF
=> operand { operand } EOF
=> ( expression ) operand EOF
=> ( abstraction ) y EOF
=> ( \ variable-list . expression ) y EOF
=> ( \ x . application ) y EOF
=> ( \ x . x ) y EOF
```